ESE5023—Assignment 01

1. Flowchart（流程图）

思路：按照流程图，首先判断 a > b。若为 True（右分支），再判断 b > c：b > c 为 True 时直接打印 a, b,c；为 False 时进入 a > c 的判断（True 打印 a, c, b, False 打印 c, a, b）。若 a > b 为 False（左分支，即 b不小于 a），再判断 b > c：b > c 为 True 时什么都不输出，直接结束；为 False 时打印 c, b,a。代码与流程图一一对应：

```
[10]: #01 Flowchat
def print_abc(a, b, c):
    if a > b:
        if b > c:
            print(a,b,c)
        else:
            if a > c:
                print(a,c,b)
            else:
                print(c,a,b)
    else:
        if b > c:
            pass
        else:
            print(c,b,a)

[11]: print_abc(4, 6, 18)
      18 6 4

[12]: print_abc(4, 18, 6)

[13]: print_abc(6,4,18)
      18 6 4

[14]: print_abc(6,18,4)

[15]: print_abc(18,4,6)
      18 6 4

[16]: print_abc(18,6,4)
      18 6 4

[18]: print_abc(3, 6, 4)
```

2.1构造矩阵

先生成一个空的大列表 M1，外层循环跑 5 次，每次先生成空的row，内层循环跑 10 次，每轮用 random.randint(0, 50)生成一个随机数，用 append 添加到 row 末尾一行填满后，把row用 append 添加进 M1。M2 同理，外层循环 10 次、内层 5 次。

2.2矩阵乘法

核心规则：结果矩阵第 i 行第 j 列的数 = M1 的第 i 行 与 M2 的第 j 列，对应位置相乘后全部加起来。用三层 for 循环实现

[1]:
```python
import random

# 做 M1：5 行 10 列
M1 = []
for i in range(5):
    row = []
    for j in range(10):
        num = random.randint(0, 50)
        row.append(num)
    M1.append(row)

# 做 M2：10 行 5 列
M2 = []
for i in range(10):
    row = []
    for j in range(5):
        num = random.randint(0, 50)
        row.append(num)
    M2.append(row)

# 2.2 矩阵乘法
def Matrix_multip(M1, M2):
    result = []                    # 结果矩阵：5 行 5 列
    for i in range(5):             # 结果的每一行
        row = []
        for j in range(5):         # 结果的每一列
            s = 0                  # 累加器，先清零
            for k in range(10):    # M1 的列数 = M2 的行数 = 10
                s = s + M1[i][k] * M2[k][j]   # 对应相乘、累加
            row.append(s)
        result.append(row)
    return result

M3 = Matrix_multip(M1, M2)
print(M3)
```

```
[[6834, 8311, 5333, 4343, 7256], [7549, 8993, 6414, 4953, 8153], [10609, 11034, 6856, 5630, 9842], [7477, 8022, 5208, 4582, 7188], [5696, 7434, 2713, 252
6, 5755]]
```

3. Pascal triangle（帕斯卡三角形）

思路：从第 1 行 [1] 开始逐行向下推导。每一行的首尾都是1，中间的每个数等于上一行"肩膀上"两个数之和。

[4]:
```
def Pascal_triangle(k):
    line = [1]
    for i in range(1,k):
        newline = [0] * (i + 1)
        newline[0] = 1
        newline[i] = 1
        for j in range(1,i):
            newline[j] = line[j-1]+line[j]
        line = newline
    print(line)

for k in range(1,6):
    print("Line",k,":")
    Pascal_triangle(k)

Pascal_triangle(100)
Pascal_triangle(200)
```

```
Line 1 :
[1]
Line 2 :
[1, 1]
Line 3 :
[1, 2, 1]
Line 4 :
[1, 3, 3, 1]
Line 5 :
[1, 4, 6, 4, 1]
[1, 99, 4851, 156849, 3764376, 71523144, 1120529256, 14887031544, 171200862756, 1731030945644, 15579278510796, 126050526132804, 924370524973896, 6186171974825304, 38000770702498296, 215337700647490344, 1130522928399324306, 5519611944537877494, 25144898858450330806, 107196674080761936594, 428786696323047746376, 1613054714739084379224, 5719012170438571889976, 19146258135816088501224, 60629817430084280253876, 181889452290252840761628, 517685364210719623706172, 1399667836569723427057428, 3599145865465003098147672, 8811701946483283447189128, 20560637875127661376774632, 45764000431735762419272568, 9724850091743849514095420 7, 45764000431735762419272568, 205606378751276613767746 32, 881170194648328344718912 8, 3599145865465003098147672, 13996678365697234 27057428, 5176853642107196237061 72, 181889452290252840761628, 60629817430084280253876, 191462581358160885501224, 571901217043857188997 6, 1613054714739084379224, 4287866963230747376, 10719667408076193659 4, 23533770064749034 4, 380007707024 98296, 6186171974825304, 924370524973896, 126050526132804, 15579278510796, 1731030945644, 171200862756, 14887031544, 1120529256, 71523144, 3764376, 156 849, 4851, 99, 1]
[1, 199, 19701, 1293699, 63391251, 2472258789, 79936367511, 2203959847089, 52895036330136, 1122550215450664, 213284540935626 16, 366461620334848584, 574 1232051912627816, 8258541490058933 8584, 109720622653640121261 6, 135322101272822816222 64, 155620416463746238656036, 167520801252150362 7885564, 169382143 48828536681954036, 161358778796735007338614764, 145222900917061506604753287 6, 123785234591209569954808324, 100153507987433197477070330076, 7707465614 68507650149628192324, 565214145076905610109727341037 6, 395649901553833927076809138726 32, 264781087962950397351403038993768, 169656030435520069414047132 3923032, 104217275838962328354343238469557 68, 614522557533191660296299785458426 32, 348229449268808607501236545093108248, 189841215891705337637770890712 0493352, 996666383431453022598297176238259009 8, 504373594039553499314895843732694711 02, 246252990031076120253743264881256829498, 116090695300364456691 0503963011639339062, 528857611923882524925896249816413476683 8, 232983218225926625845732672216419991079 62, 993244246121055615447597181554211540918 38, 410 0315990397178309924696057185334822765 62, 164012639615887132396987842287413392910624 8, 636049017046976928076123583504847060311 95, 239275582603599365865 7208395699442465545918 48, 873634103923992783934458561042150397458355 2, 309743000482142896122217126187671504553416248, 106689255721626997553208121242 01849017322632, 357177073502838209199870666768102358149275768, 116272536693477119165064280884084384674124 53032, 368196366196010877356036889466267218 13473401268, 113464594480811515266860347570217040690499665132, 340393783442434545800581042710651122071498995396, 994483798684759751456599516938961121461 44123804, 283045388856431621568416785590319703767748712159 6, 785050418148970723972778631731641442525642654484, 212254372314351343888936444875591945571 74782880396, 559579708828744452070832445581106038325517003210 44, 143891925127391430532499771720855838426561515111256, 559579708828744452070832445581106038325517003210 44, 212254372314351343888936444875591945571 4782880396, 785050418148970723972778631731641442525642654804, 283045388856431621568416785590319703767748712159 6, 994483798684759751456599516938961121 3 46144123804, 340393783442434545800581042710651122071498995396, 113464594480811515266860347570217040690499665132, 368196366196010877356036889466267218 13 473401268, 116272536693477119165064280884084384674126530 32, 357177073502838209199870666768102358149275768, 106689255721626997553208121242 0184901732 2632, 309743000482142896122217126187671504553416248, 873634103923992783934458561042150397458355 2, 239275582603386558657208395699442465545918 48, 636049017046976928076123583504847060311 952, 239275582603386558657208395699442465545918 48, 636049017046976928076123583504847060311 95, 164012639615887132396987842287413392910624 8, 412031599039717830992469605718533482765 62
```

4. Add or double（翻倍或加一）

思路（倒推法）：从目标金额x倒着往回走。正向操作是"×2"或"+1"，那么倒推时：如果x是偶数，上一步一定是"×2"来的（x/2）；如果x是奇数，上一步一定是"+1"来的（x-1）。每倒推一步计数加1，直到回到1。这样每一步都是"必须做且只需做"的，所以步数最少。

[2]:
```
import random

def Least_moves(x):
    moves=0
    while x > 1:
        if x % 2 == 0:
            x = x // 2
        else:
            x = x - 1
        moves = moves + 1
    print(moves)

Least_moves(2)
Least_moves(5)
```

```
1
3
```

[7]:
```
import random

def Least_moves(x):
    moves=0
    while x > 1:
        if x % 2 == 0:
            x = x // 2
        else:
            x = x - 1
        moves = moves + 1
    print(moves)

Least_moves(2)
Least_moves(5)

x = random.randint(1,100)
print("x =",x)
Least_moves(x)
```

```
1
3
x = 66
7
```

5. 在 123456789 中插入+或-

思路：9 个数字之间有 8 个空隙，每个空隙有3种选择：插入 "+"、插入 "-"、或者什么都不插（数字相连组成多位数）。所以一共有 3^8 = 6561 种可能的表达式。用枚举所有组合，逐个用 eval 计算，把结果等于目标值的表达式打印出来。

[1]:
```python
from itertools import product

def all_solutions(x):
    solutions = []
    digits = "123456789"
    for ops in product(["+", "-", ""], repeat=8):
        expr = digits[0]
        for d, op in zip(digits[1:], ops):
            expr = expr + op + d
        if eval(expr) == x:
            solutions.append(expr)
    return solutions

def Find_expression(x):
    for expr in all_solutions(x):
        print(expr + "=" + str(x))

Find_expression(50)
```

```
1+2+3+4-56+7+89=50
1+2+3-4+56-7+8-9=50
1+2+34-5-6+7+8+9=50
1+2+34-56+78-9=50
1+2-3+4+56+7-8-9=50
1+2-34+5-6-7+89=50
1-2+3-45+6+78+9=50
1-2+34+5+6+7+8-9=50
1-2+34-5-67+89=50
1-2-3+4+56-7-8+9=50
1-2-3-4-5-6+78-9=50
1-2-34-5-6+7+89=50
1-23+4+5-6+78-9=50
1-23-4-5-6+78+9=50
12+3+4-56+78+9=50
12-3+45+6+7-8-9=50
12-3-4-5+67-8-9=50
```

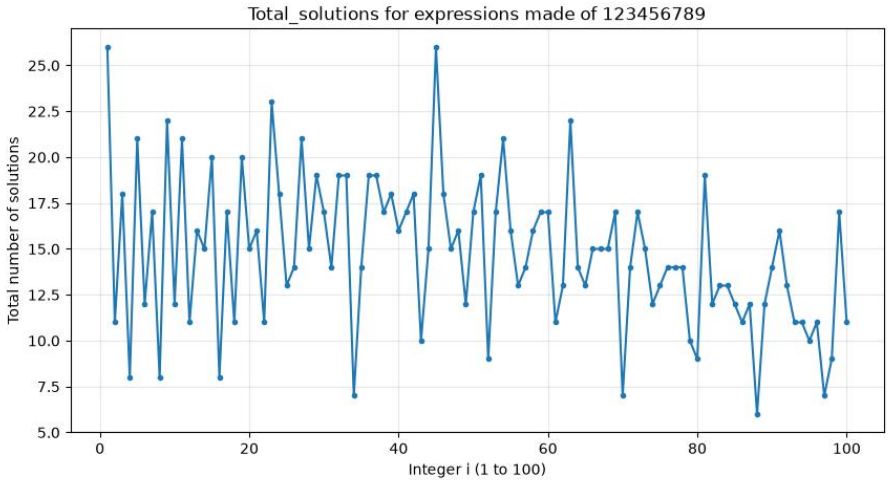
[4]:
```python
from itertools import product
import matplotlib.pyplot as plt

def all_solutions(x):
    solutions = []
    digits = "123456789"
    for ops in product(["+", "-", ""], repeat=8):
        expr = digits[0]
        for d, op in zip(digits[1:], ops):
            expr = expr + op + d
        if eval(expr) == x:
            solutions.append(expr)
    return solutions

Total_solutions = []
for i in range(1, 101):
    Total_solutions.append(len(all_solutions(i)))

plt.figure(figsize=(10, 5))
plt.plot(range(1, 101), Total_solutions, marker="o", markersize=3)
plt.xlabel("Integer i (1 to 100)")
plt.ylabel("Total number of solutions")
plt.title("Total_solutions for expressions made of 123456789")
plt.grid(True, alpha=0.3)
plt.show()

maximum = max(Total_solutions)
minimum = min(Total_solutions)
print("Maximum =", maximum, "at i =", [i + 1 for i in range(100) if Total_solutions[i] == maximum])
print("Minimum =", minimum, "at i =", [i + 1 for i in range(100) if Total_solutions[i] == minimum])
```




```
Maximum = 26 at i = [1, 45]
Minimum = 6 at i = [88]
```